Object Oriented Design Patterns - II

- Design Patterns
  - Decorator Design Pattern
  - Composite Design Pattern
  - Bridge Design Pattern
  - Builder Design Pattern
  - Chain of Responsibility Design Pattern
  - Flyweight Design Pattern
  - Interpreter Design Pattern
  - Mediator Design Pattern
  - Prototype Design Pattern
  - Memento Design Pattern
  - Visitor Design Pattern
- Compound Patterns
  - What is a Compound Pattern?
  - What is MVC (Model-View-Controller)?
    - How MVC works in Java EE?
- Anti-Patterns
  - What are Design patterns
  - What are Anti-Patterns
    - Anti-Pattern 1: Spaghetti Code
    - Anti-Pattern 2: God Object
    - Anti-Pattern 3: Copy-Paste Programming
    - Anti-Pattern 4: Long Method

Anti-Pattern 5: Tight Coupling

Anti-Pattern 6: Primitive Obsession

Anti-Pattern 7: Magic Numbers and Strings

Anti-Pattern 8: Swallowed Exceptions

Design Patterns

Decorator Design Pattern

The **Decorate Design Pattern** is a structural design pattern that **allows you to add new features to behaviors to an object dynamically without modifying its original class.**

- **"I need to add extra functionality to an object." → Wrap the object with a decorator.**
- **"I need to combine multiple optional features." → Stack multiple decorators around the same object.**

```
interface IceCream {
    public String getDescription();
    public double getPrice();
}

class VanillaIceCream implements IceCream {

    @Override
    public String getDescription() {
        return "Vanilla Ice Cream";
    }

    @Override
    public double getPrice() {
        return 100;
    }
}
```

```
abstract class IceCreamDecorator implements IceCream {
    protected IceCream iceCream;

    public IceCreamDecorator(IceCream iceCream) {
        this.iceCream = iceCream;
    }
}

class ChocolateTopping extends IceCreamDecorator {

    public ChocolateTopping(IceCream iceCream) {
        super(iceCream);
    }

    @Override
    public String getDescription() {
        return this.iceCream.getDescription() + " with Chocolate Topping";
    }

    @Override
    public double getPrice() {
        return this.iceCream.getPrice() + 50;
    }
}

class NutsTopping extends IceCreamDecorator {

    public NutsTopping(IceCream iceCream) {
        super(iceCream);
    }

    @Override
    public String getDescription() {
        return this.iceCream.getDescription() + " with Nuts Topping";
    }

    @Override
    public double getPrice() {
```

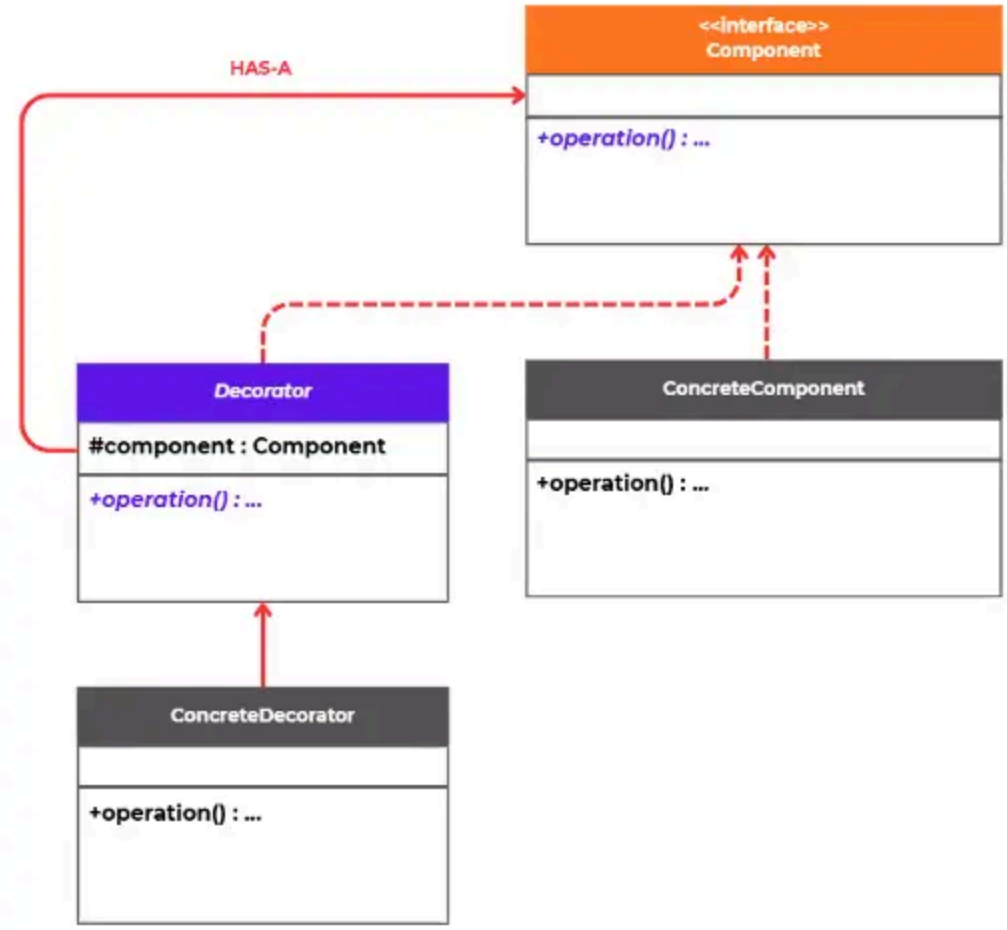
```
        return this.iceCream.getPrice() + 25;
    }
}
```

```
public class Test {

    public static void main(String[] args) {
        IceCream iceCream1 = new VanillaIceCream();
        System.out.println(iceCream1.getDescription());
        System.out.println(iceCream1.getPrice());

        IceCream iceCream2 = new VanillaIceCream();
        iceCream2 = new ChocolateTopping(iceCream2);
        System.out.println(iceCream2.getDescription());
        System.out.println(iceCream2.getPrice());

        iceCream2 = new NutsTopping(iceCream2);
        System.out.println(iceCream2.getDescription());
        System.out.println(iceCream2.getPrice());
    }
}
```



Decorator Design Pattern - Class Diagram

Composite Design Pattern

The **Composite Design Pattern** is a structural design pattern that **allows you to treat individual objects and groups of objects uniformly through a common interface.**

- **"I need to treat a single object and a group of objects in the same way." → Create a common component interface.**
- **"I need to organize objects into a tree-loke structure." → Use composite objects that contain individual objects or other composites.**

```java
interface Component {
    public void display();
    public double getSize();
}

class File implements Component {

    private String name;
    private double size;

    public File(String name, double size) {
        this.name = name;
        this.size = size;
    }

    @Override
    public void display() {
        System.out.println("File: " + this.name + ", Size: " + this.size + "Mb");
    }

    @Override
    public double getSize() {
        return this.size;
    }
}

class Folder implements Component {

    private String name;
    private ArrayList<Component> componentArrayList = new ArrayList<>();

    public Folder(String name) {
        this.name = name;
    }

    public void add(Component component) {
        componentArrayList.add(component);
    }
```

```
    public void remove(Component component) {
        componentArrayList.remove(component);
    }

    @Override
    public void display() {
        System.out.println("Folder: " + this.name + ", Size: " + this.getSize() +
"Mb");
        for (Component component : componentArrayList) {
            component.display();
        }

    }

    @Override
    public double getSize() {

        double total = 0;

        for (Component component : componentArrayList) {
            total += component.getSize();
        }

        return total;
    }
}
```

```
public class Test {

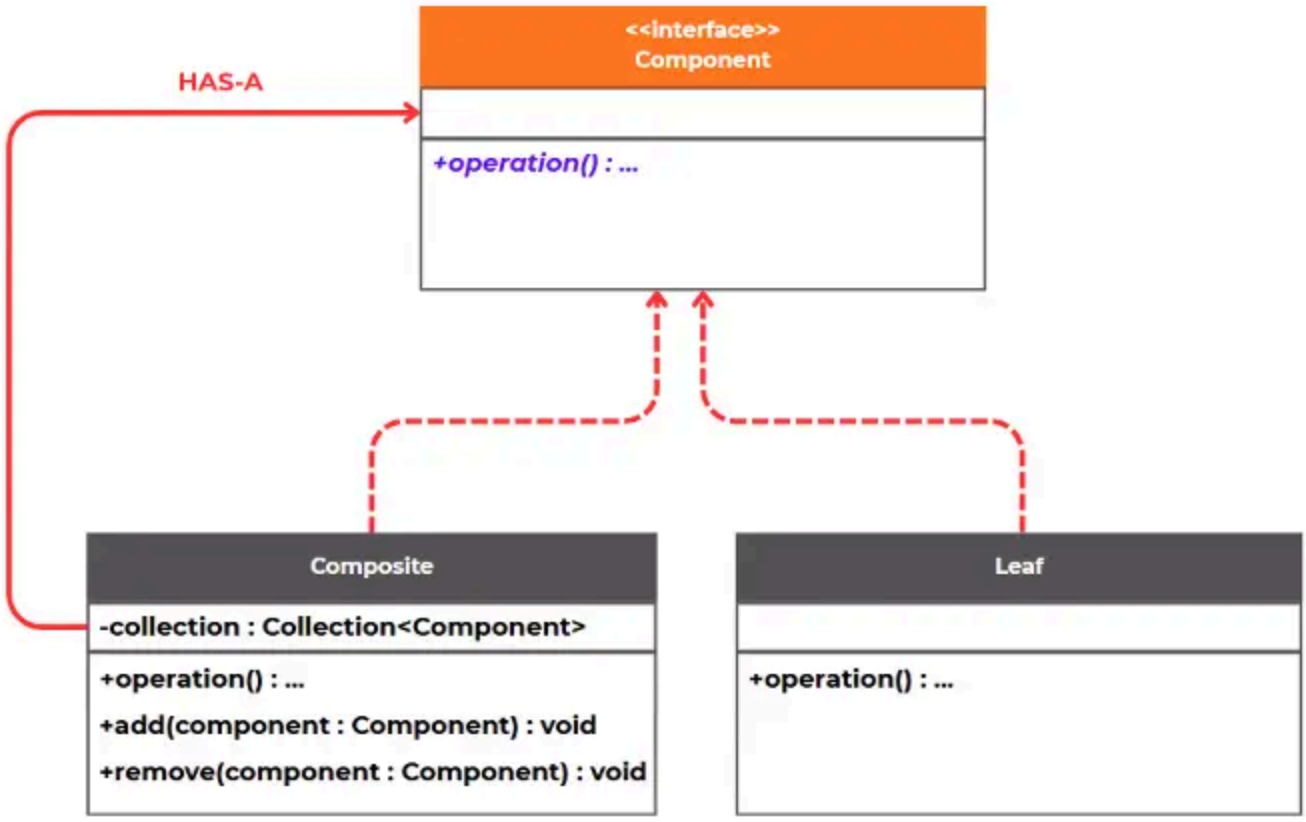
    public static void main(String[] args) {
        Folder folder1 = new Folder("Desktop");

        Folder folder2 = new Folder("Documents");
        folder1.add(folder2);
        File file1 = new File("Game.exe", 2048);
        folder1.add(file1);
        Folder folder3 = new Folder("Images");
        folder1.add(folder3);

        File file2 = new File("Note.txt", 1.5);
        folder2.add(file2);
        File file3 = new File("Report.pdf", 4);
        folder2.add(file3);

        File file4 = new File("Photo.jpg", 8.2);
        folder3.add(file4);

        folder1.display();
    }
}
```



Composite Design Pattern - Class Diagram

Bridge Design Pattern

The **Bridge Design Pattern** is a structural pattern that **allows you to separate an abstraction from its implementation so that both can vary independently.**

- **"I need one feature to work with different implementations." → Separate the abstraction from the implementation using a bridge.**
- **"I want to add new types and implementation without creating many subclasses." → Use composition to connect the abstraction to an implementation interface.**

```
interface OS {
    public void launchApp();
    public void closeApp();
}

abstract class App {

    protected OS os;

    public App(OS os) {
        this.os = os;
    }

    public abstract void run();
    public abstract void exit();
}
```

```
class Windows implements OS {

    @Override
    public void launchApp() {
        System.out.println("Windows: App Launching");
    }

    @Override
    public void closeApp() {
        System.out.println("Windows: App Closing");
    }
}

class MacOS implements OS {
    @Override
    public void launchApp() {
        System.out.println("MacOS: App Launching");
    }

    @Override
    public void closeApp() {
        System.out.println("MacOS: App Closing");
    }
}
```

```
class TextEditor extends App {

    public TextEditor(OS os) {
        super(os);
    }

    @Override
    public void run() {
        System.out.println("Text Editor Running...");
        os.launchApp();
    }

    @Override
    public void exit() {
        System.out.println("Text Editor Closing...");
        os.closeApp();
    }
}

class MediaPlayer extends App {

    public MediaPlayer(OS os) {
        super(os);
    }

    @Override
    public void run() {
        System.out.println("Media Player Running...");
        os.launchApp();
    }

    @Override
    public void exit() {
        System.out.println("Media Player Closing...");
        os.closeApp();
    }
}
```

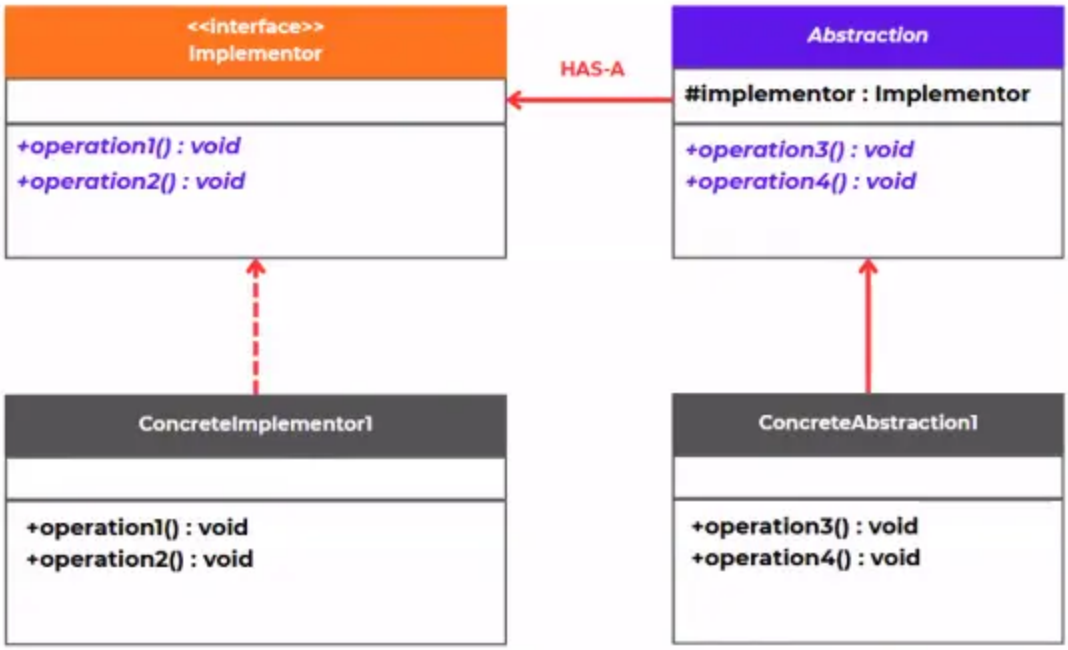
```
public class Test {

    public static void main(String[] args) {

        OS os1 = new Windows();
        OS os2 = new MacOS();

        App app1 = new TextEditor(os1);
        app1.run();
        app1.exit();

        App app2 = new TextEditor(os2);
        app2.run();
        app2.exit();
    }
}
```



Bridge Design Pattern - Class Diagram

Builder Design Pattern

The **Builder Design Pattern** is a creational design pattern that **allows you to construct complex objects step by step, separating the construction process from the final project.**

- **"I need to create an object with many optional properties." → Use a builder to set the properties step by step.**
- **"I need different versions of the same object." → Use the same building process with different configurations.**

```
class Computer {
    private String cpu;
    private int ram;
    private int storage;
    private String gpu;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.gpu = builder.gpu;
    }

    @Override
    public String toString() { // Optional
        return "CPU: " + this.cpu + "\n" +
                "RAM: " + this.ram + "GB\n" +
                "STORAGE: " + this.storage + "\n" +
                "GPU: " + this.gpu;
    }

    public static class Builder {
        private String cpu;
        private int ram;
        private int storage;
        private String gpu;

        public Builder(String cpu) {
            this.cpu = cpu;
        }

        public Builder setRam(int ram) {
            this.ram = ram;
            return this; // Method Chaining
        }

        public Builder setStorage(int storage) {
            this.storage = storage;
```

```
            return this; // Method Chaining
        }

        public Builder setGpu(String gpu) {
            this.gpu = gpu;
            return this; // Method Chaining
        }

        public Computer build() {

            if (this.cpu == null) {
                System.out.println("CPU must be specified!");
            }

            return new Computer(this);
        }
    }
}
```

```
public class Test {

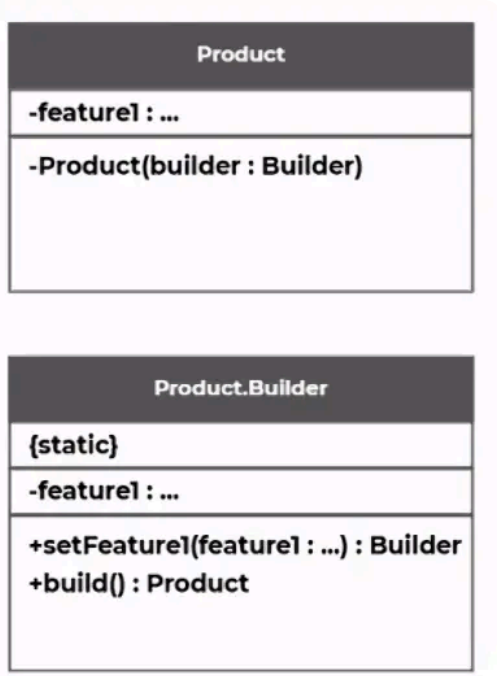
    public static void main(String[] args) {

        //Builder builder = new Builder();

        //Computer computer = new Computer();

        Computer computer = new Computer.Builder("Intel Core i7")
                .setRam(16)
                .setStorage(512)
                .setGpu("Nvidia RTX 5070")
                .build();

        System.out.println(computer);
    }
}
```



Builder Design Pattern - Class Diagram

Chain of Responsibility Design Pattern

The **Chain of Responsibility (CoR)** pattern is a behavioral design pattern. It lets **a request pass through a chain of potential handlers**, where each handler decides either:

- **"I can handle this request." → handle it, or**
- **"This isn't my responsibility." → pass it to the next handler.**

```
class Patient {
    private String name;
    private String symptom;

    public Patient(String name, String symptom) {
        this.name = name;
        this.symptom = symptom;
    }

    public String getName() {
        return name;
    }

    public String getSymptom() {
        return symptom;
    }
}
```

```
abstract class Doctor {
    private Doctor doctor;

    public Doctor getDoctor() {
        return doctor;
    }

    public void setDoctor(Doctor doctor) {
        this.doctor = doctor;
    }

    public abstract void handle(Patient patient);
}
```

```
class GeneralDoctor extends Doctor {
    @Override
    public void handle(Patient patient) {
        if (patient.getSymptom().equalsIgnoreCase("Fever")) {
            System.out.println("General Doctor Treating " + patient.getName());
        } else {
            this.getDoctor().handle(patient);
        }
    }
}

class Specialist extends Doctor {
    @Override
    public void handle(Patient patient) {
        if (patient.getSymptom().equalsIgnoreCase("Chest Pain")) {
            System.out.println("Specialist Treating " + patient.getName());
        } else {
            this.getDoctor().handle(patient);
        }
    }
}

class Consultant extends Doctor {
    @Override
    public void handle(Patient patient) {
        if (patient.getSymptom().equalsIgnoreCase("Heart Problem")) {
            System.out.println("Consultant Treating " + patient.getName());
        } else {
            System.out.println("Can't Handle...");
        }
    }
}
```

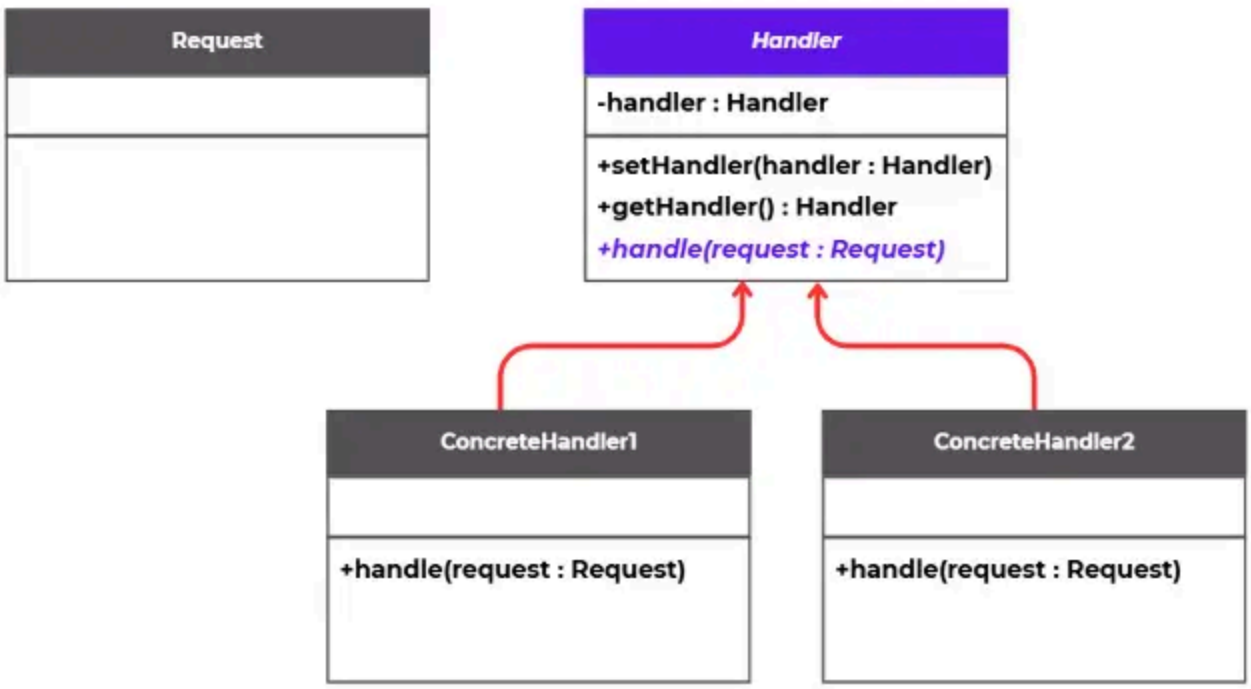
```
public class Test {

    public static void main(String[] args) {
        Patient patient = new Patient("Saman", "Heart Problem");

        GeneralDoctor generalDoctor = new GeneralDoctor();
        Specialist specialist = new Specialist();
        Consultant consultant = new Consultant();

        generalDoctor.setDoctor(specialist);
        specialist.setDoctor(consultant);

        generalDoctor.handle(patient);
    }
}
```

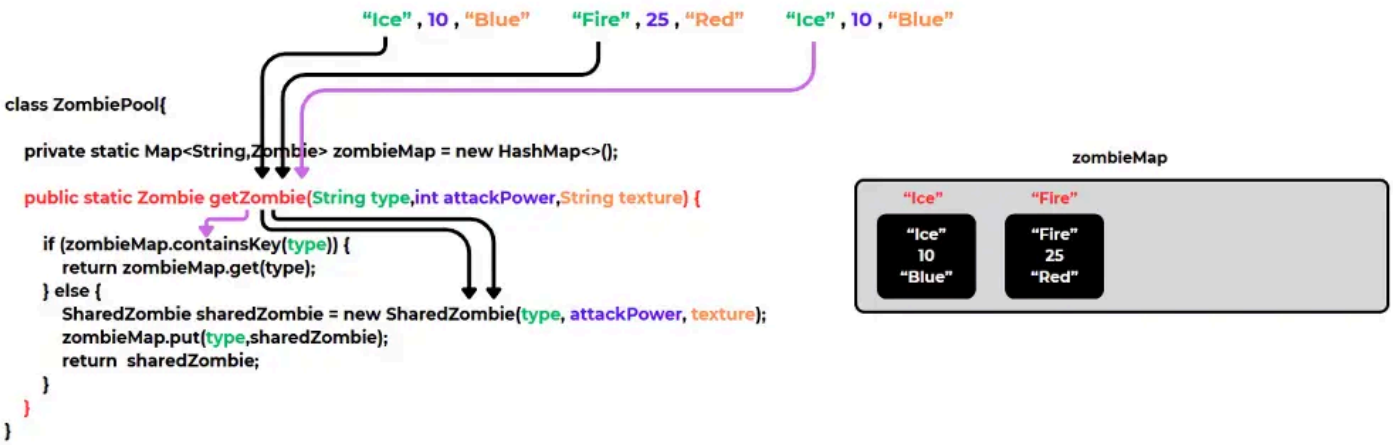


Chain of Responsibility Design Pattern - Class Diagram

Flyweight Design Pattern

The **Flyweight Design Pattern** is a structural design pattern. **It reduces memory usage by sharing common objects**, instead of creating a separate copy of the same data for every object.

- **"This data is common to many objects." → store it once and share it.**
- **This data is unique to this object." → keep it outside the shared object.**



```
abstract class AbstractZombie {
    public abstract void display();
}

class SharedZombie {
    private String type;
    private int attackPower;
    private String texture;

    public SharedZombie(String type, int attackPower, String texture) {
        this.type = type;
        this.attackPower = attackPower;
        this.texture = texture;
    }

    public String getType() {
        return type;
    }

    public int getAttackPower() {
        return attackPower;
    }

    public String getTexture() {
        return texture;
    }
}
```

```
class SharedZombiePool {

    private static final ArrayList<SharedZombie> sharedZombieArrayList = new
ArrayList<>();

    public static SharedZombie getSharedZombie(String type, int attackPower, String
texture) {
        SharedZombie sz = null;
        for (SharedZombie szItem : sharedZombieArrayList) {

            if (type.equals(szItem.getType()) && attackPower ==
szItem.getAttackPower() && texture.equals(szItem.getTexture())) {
                sz = szItem;
                break;
            }
        }

        if (sz == null) {
            sz = new SharedZombie(type, attackPower, texture);
            sharedZombieArrayList.add(sz);
        }

        return sz;
    }
}
```

```
class Zombie extends AbstractZombie {
    private int x;
    private int y;
    private int health;

    private SharedZombie sharedZombie;

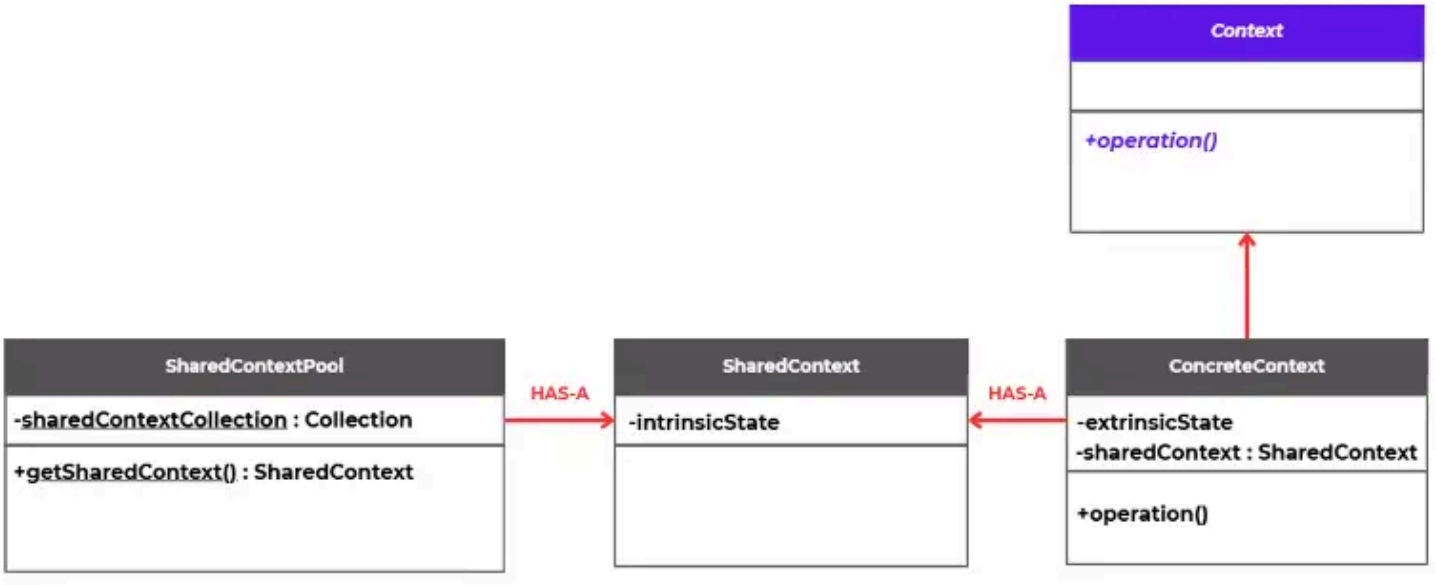
    public Zombie(String type, int attackPower, String texture, int x, int y, int
health) {
        this.x = x;
        this.y = y;
        this.health = health;
        this.sharedZombie = SharedZombiePool.getSharedZombie(type, attackPower,
texture);
    }

    @Override
    public void display() {
        System.out.println("Type: " + this.sharedZombie.getType());
        System.out.println("Attack Power: " + this.sharedZombie.getAttackPower());
        System.out.println("Texture: " + this.sharedZombie.getTexture());
        System.out.println("Position x: " + this.x + " y: " + this.y);
        System.out.println("Health: " + this.health);
        //System.out.println(this.sharedZombie);
        System.out.println();
    }
}

public class Test {
    public static void main(String[] args) {
        Zombie z1 = new Zombie("Ice", 10, "Blue", 250, 250, 100);
        Zombie z2 = new Zombie("Fire", 25, "Red", 500, 250, 100);
        Zombie z3 = new Zombie("Ice", 10, "Blue", 500, 250, 90);
        Zombie z4 = new Zombie("Ice", 10, "Blue", 750, 250, 100);

        z1.display();
        z2.display();
        z3.display();
```

```
            z4.display();
        }
    }
```



Flyweight Design Pattern - Class Diagram

Interpreter Design Pattern

The **Interpreter Design Pattern** is a behavioral pattern. It defines way to represent a language's grammar and **provides a mechanism to interpret expressions** written in that language.

- **"This expression follows a known grammar." → Represent it using object and interpret it.**
- **"This expression contains smaller expressions." → Combine interpreters to evaluate the whole expression.**

For example, a calculator could interpret expressions such as **5 + 3** or **(10 - 2) * 4**

```
interface Expression {
    public int interpret();
}

class TerminalExpression implements Expression {

    private int value;

    public TerminalExpression(int value) {
        this.value = value;
    }

    @Override
    public int interpret() {
        return this.value;
    }
}

class NonTerminalExpression implements Expression {

    private Expression e1;
    private Expression e2;

    public NonTerminalExpression(Expression e1, Expression e2) {
        this.e1 = e1;
        this.e2 = e2;
    }

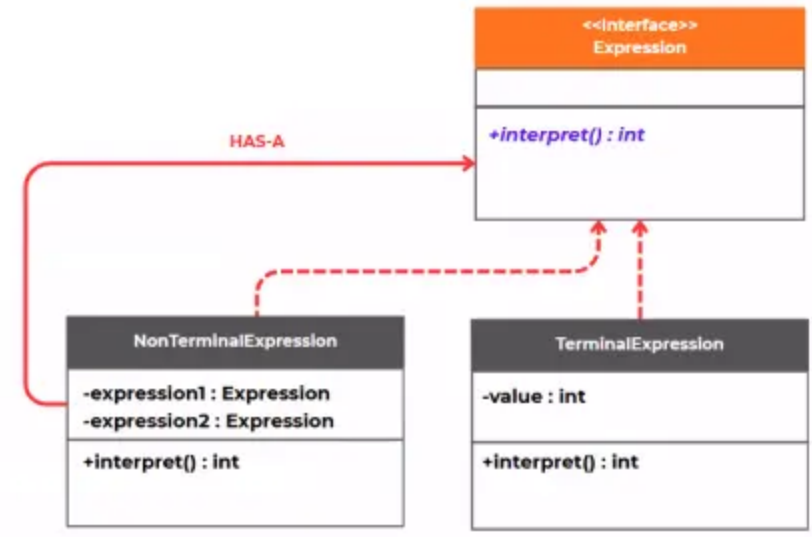
    @Override
    public int interpret() {
        return e1.interpret() + e2.interpret();
    }
}
```

```
public class Test {
    public static void main(String[] args) {

        Expression e1 = new NonTerminalExpression(
                new TerminalExpression(10),
                new TerminalExpression(20)
        );

        Expression e2 = new NonTerminalExpression(
                new NonTerminalExpression(
                        new TerminalExpression(10),
                        new TerminalExpression(20)
                ),
                new TerminalExpression(50)
        );

        System.out.println(e1.interpret());
        System.out.println(e2.interpret());
    }
}
```



Interpreter Pattern - Class Diagram

Mediator Design Pattern

The **Mediator Design Pattern** is behavioral design pattern that **centralizes communication between multiple objects through a mediator object**, instead of allowing those objects to communicate directly with each other.

- **"Objects need to communicate with each other." → Let them communicate through a mediator.**
- **"Many objects depend on one another." → Reduce direct dependencies by centralizing communication.**

```
interface AirTrafficControl {
    public void requestTakeOff(Flight f);

    public void requestLanding(Flight f);

    public void notifyFlight(String message, Flight f);
}

class AirportControlTower implements AirTrafficControl {

    @Override
    public void requestTakeOff(Flight f) {
        System.out.println("Flight: " + f.getNumber() + " request permission to
take-off");
        notifyFlight("Take-off approved", f);
    }

    @Override
    public void requestLanding(Flight f) {
        System.out.println("Flight: " + f.getNumber() + " request permission to
landing");
        notifyFlight("Landing approved", f);
    }

    @Override
    public void notifyFlight(String message, Flight f) {
        f.receiveMessage(message);
    }
}
```

```
abstract class Flight {

    private AirTrafficControl airTrafficControl;
    private String number;

    public Flight(AirTrafficControl airTrafficControl, String number) {
        this.airTrafficControl = airTrafficControl;
        this.number = number;
    }

    public String getNumber() {
        return number;
    }

    public void requestTakeOff() {
        airTrafficControl.requestTakeOff(this);
    }

    public void requestLanding() {
        airTrafficControl.requestLanding(this);
    }

    public abstract void receiveMessage(String message);
}

class FlightA extends Flight {

    public FlightA(AirTrafficControl airTrafficControl, String number) {
        super(airTrafficControl, number);
    }

    @Override
    public void receiveMessage(String message) {
        System.out.println("Flight A: " + getNumber() + " New message: " + message);
    }
}

class FlightB extends Flight {
```

```
    public FlightB(AirTrafficControl airTrafficControl, String number) {
        super(airTrafficControl, number);
    }

    @Override
    public void receiveMessage(String message) {
        System.out.println("Flight B: " + getNumber() + " New message: " + message);
    }
}
```

```
public class Test {

    public static void main(String[] args) {

        AirportControlTower airportControlTower = new AirportControlTower();

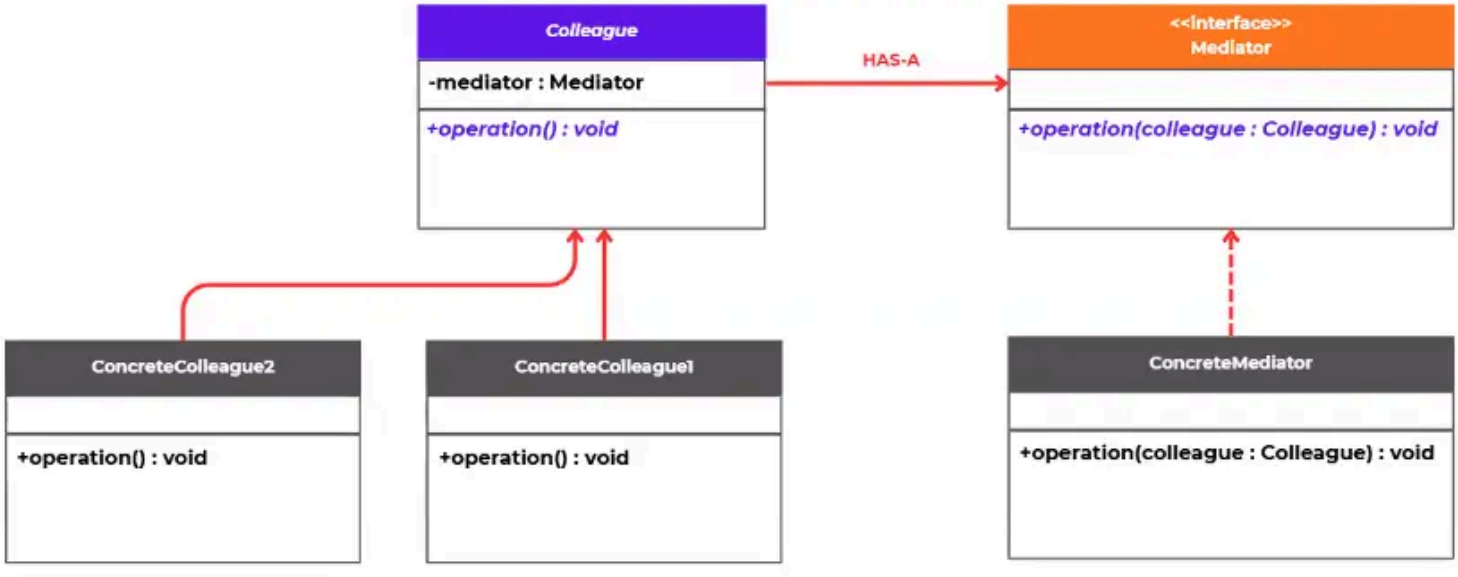
        Flight fa1 = new FlightA(airportControlTower, "123");

        Flight fb1 = new FlightB(airportControlTower, "456");
        Flight fb2 = new FlightB(airportControlTower, "789");

        fa1.requestLanding();

        fb1.requestLanding();

        fb2.requestTakeOff();
    }
}
```



Mediator Design Pattern - Class Diagram

Prototype Design Pattern

The **Prototype Design Pattern** is a creational design pattern that **creates new objects by cloning existing objects**, rather than constructing every object from scratch.

- **"Creating an object from scratch is expensive or complicated." → Clone an existing object.**
- **"I need another object with similar properties." → Copy the prototype and modify the copy as needed.**

🌟 Java clone method

```
class Gun implements Cloneable {
    int bullets = 10;

    public void fire() {
        System.out.println("Trrr...");
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class Test {

    public static void main(String[] args) {
        Gun g1 = new Gun();
        g1.fire();

        //Gun g2 = new Gun();
        try {
            Gun g2 = (Gun) g1.clone();
            g2.fire();
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
}
```

```
interface Prototype {
    public Prototype clone();
}

class Zombie implements Prototype {

    private String type;
    private int attackPower;
    private int health;

    public Zombie(String type, int attackPower, int health) {
        this.type = type;
        this.attackPower = attackPower;
        this.health = health;
    }

    public void setType(String type) {
        this.type = type;
    }

    public void setAttackPower(int attackPower) {
        this.attackPower = attackPower;
    }

    public void setHealth(int health) {
        this.health = health;
    }

    public void display() {
        System.out.println("Type: " + this.type);
        System.out.println("Attack Power: " + this.attackPower);
        System.out.println("Health: " + this.health);
        System.out.println();
    }

    @Override
    public Zombie clone() {
        return new Zombie(this.type, this.attackPower, this.health);
```

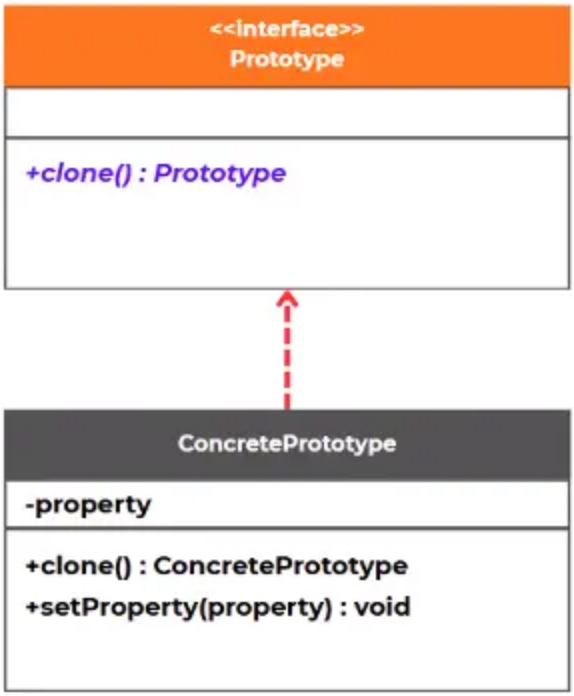
```
    }
}
```

```
public class Test {

    public static void main(String[] args) {
        Zombie z1 = new Zombie("Ice Zombie", 50, 100);

        //Zombie z2 = new Zombie("Fire Zombie", 80, 100);
        Zombie z2 = z1.clone();
        z2.setType("Fire Zombie");
        z2.setAttackPower(80);

        z1.display();
        z2.display();
    }
}
```



Prototype Design Pattern - Class Diagram

Memento Design Pattern

The **Memento Design Pattern** is a behavioral design pattern that allows **an object to save its internal state and restore it later** without exposing its internal implementation.

- **"I need to save the current state of an object." → Create a memento containing that state.**
- **"I need to undo a change." → restore the previously saved state.**

Originator

- The object whose state needs to be saved and restored

Memento

- Stores a snapshot of the originator's state.

Caretaker

- Keeps saved mementos but does not modify their internal state

```
import java.util.Stack;

class Player {
    private String name;
    private int health;
    private int score;

    public Player(String name, int health, int score) {
        this.name = name;
        this.health = health;
        this.score = score;
    }

    public String getName() {
        return name;
    }

    public int getHealth() {
        return health;
    }

    public int getScore() {
        return score;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setHealth(int health) {
        this.health = health;
    }

    public void setScore(int score) {
        this.score = score;
    }

    public void display() {
```

```
        System.out.println("Player Name: " + this.name);
        System.out.println("Health: " + this.health);
        System.out.println("Score: " + this.score);
    }
}
```

```
final class PlayerMemento {

    private int score;

    public PlayerMemento(int score) {
        this.score = score;
    }

    public int getScore() {
        return score;
    }
}

class GameCaretaker {

    private Stack<PlayerMemento> playerMementoStack = new Stack<>();

    public void saveGame(Player player) {
        PlayerMemento playerMemento = new PlayerMemento(player.getScore());
        playerMementoStack.push(playerMemento);
        System.out.println("Game Saved Successfully!");
    }

    public void loadGame(Player player) {
        PlayerMemento playerMemento = playerMementoStack.pop();
        player.setScore(playerMemento.getScore());
        System.out.println("Game Loaded Successfully!");
    }
}
```

```
public class Test {

    public static void main(String[] args) {

        Player player = new Player("Mike", 100, 0);

        GameCaretaker gameCaretaker = new GameCaretaker();
        gameCaretaker.saveGame(player);

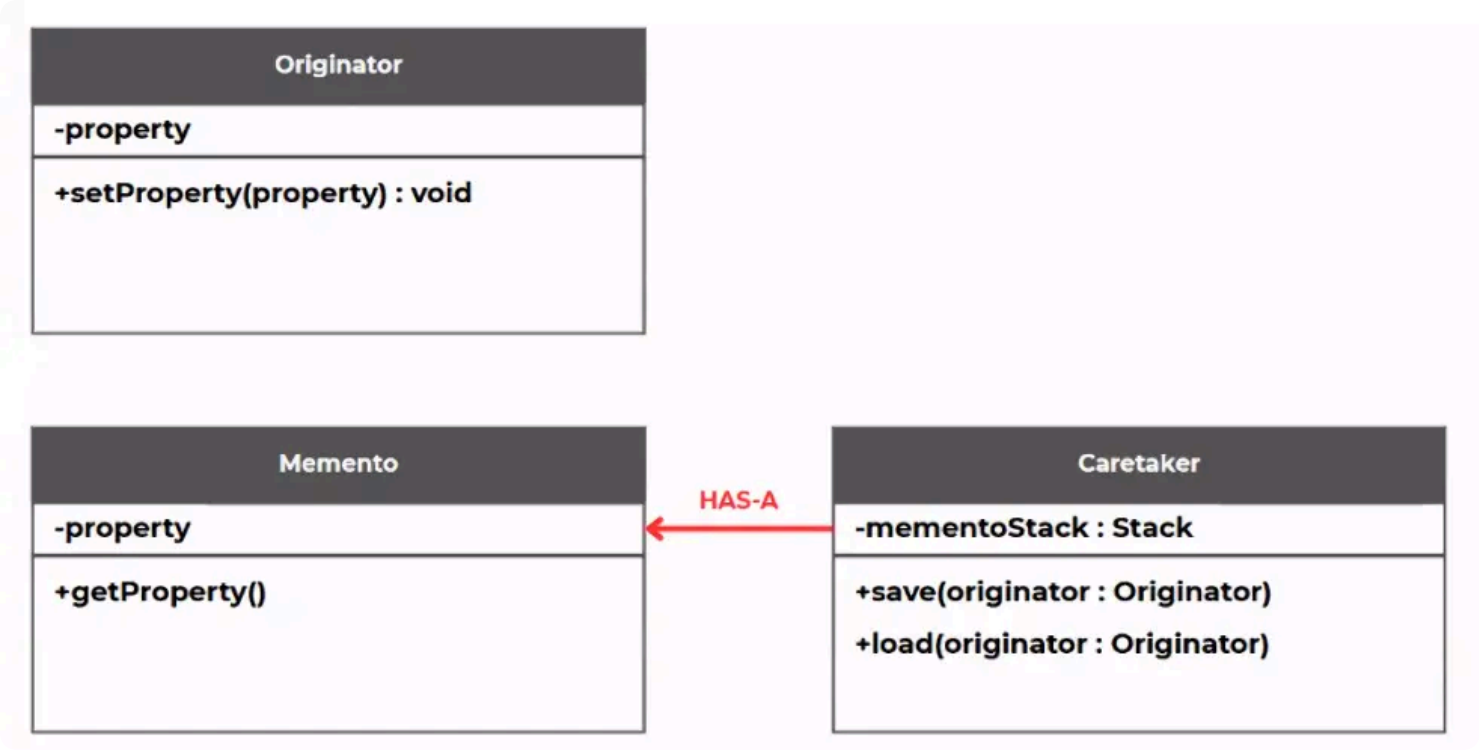
        player.setScore(100);
        gameCaretaker.saveGame(player);

        player.setScore(250);
        gameCaretaker.saveGame(player);

        player.setScore(600);
        player.display();

        gameCaretaker.loadGame(player);
        player.display();

        gameCaretaker.loadGame(player);
        player.display();
    }
}
```



Memento Design Pattern - Class Diagram

Visitor Design Pattern

The **Visitor Design Pattern** is a behavioral design pattern that **allows you to add new operations to existing objects without modifying their classes.**

- **"I need to perform a new operation on different types of objects." → Create a visitor that defines the operation for each object type.**
- **"I need to add another operation without changing the existing classes." → Create a new visitor implementation.**

```
interface Visitor {
    void visit(Lion lion);
    void visit(Elephant elephant);
    void visit(Dog dog);
}

interface Animal {
    void run();
    void accept(Visitor visitor);
}
```

```
class EatVisitor implements Visitor {

    @Override
    public void visit(Lion lion) {
        System.out.println("Lion-eat");
    }

    @Override
    public void visit(Elephant elephant) {
        System.out.println("Elephant-eat");
    }

    @Override
    public void visit(Dog dog) {
        System.out.println("Dog-eat");
    }
}
```

```
class Lion implements Animal {

    @Override
    public void run() {
        System.out.println("Lion-run");
    }

    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}

class Elephant implements Animal {

    @Override
    public void run() {
        System.out.println("Elephant-run");
    }

    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}

class Dog implements Animal {

    @Override
    public void run() {
        System.out.println("Dog-run");
    }

    @Override
    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
```

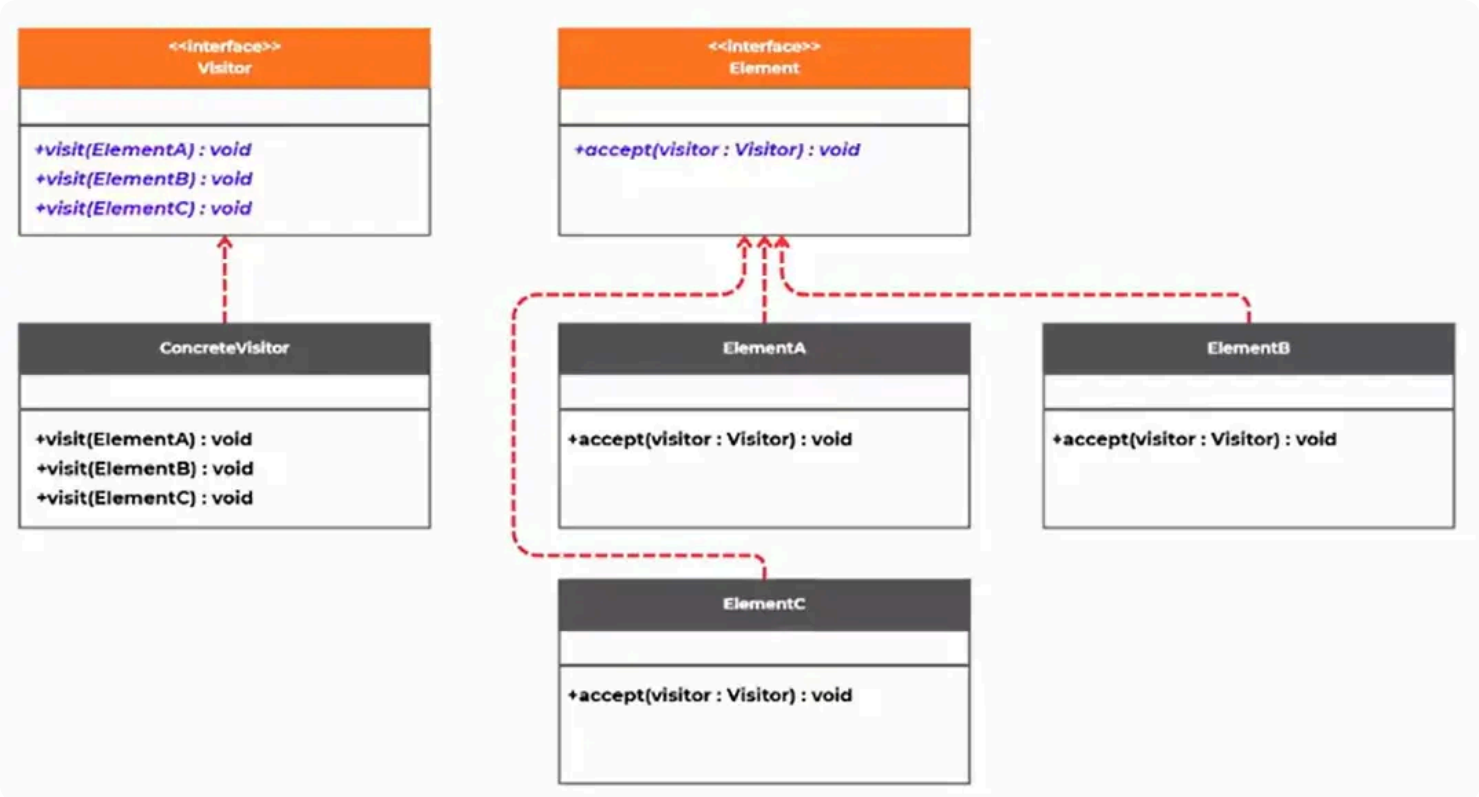
```
}
```

```
public class Test {

    public static void main(String[] args) {
        Lion lion = new Lion();
        Elephant elephant = new Elephant();
        Dog dog = new Dog();

        lion.run();
        elephant.run();
        dog.run();

        EatVisitor eatVisitor = new EatVisitor();
        lion.accept(eatVisitor);
        elephant.accept(eatVisitor);
        dog.accept(eatVisitor);
    }
}
```



Visitor Design Pattern - Class Diagram

Compound Patterns

What is a Compound Pattern?

A Compound Pattern **combines two or more design patterns to solve a larger or more complex software design problem.**

- **"One design pattern is not enough to solve the whole problem." → Combine multiple patterns.**
- **"Different parts of a system have different responsibilities." → Use appropriate patterns for each responsibility**

What is MVC (Model-View-Controller)?

Model-View-Controller (MVC) is an **architectural pattern that separates an application into three responsibilities: managing data, displaying the user interface, and handling use input.**

- **"I need to manage application data." → Use the Model.**
- **"I need to display information to the user." → Use the View.**
- **"I need to handle user actions and coordinate the application." → Use the controller.**

How MVC works in Java EE?

Step 1 - Browser

The user visits/student or submits the marks form

Step 2 - Servlet Controller

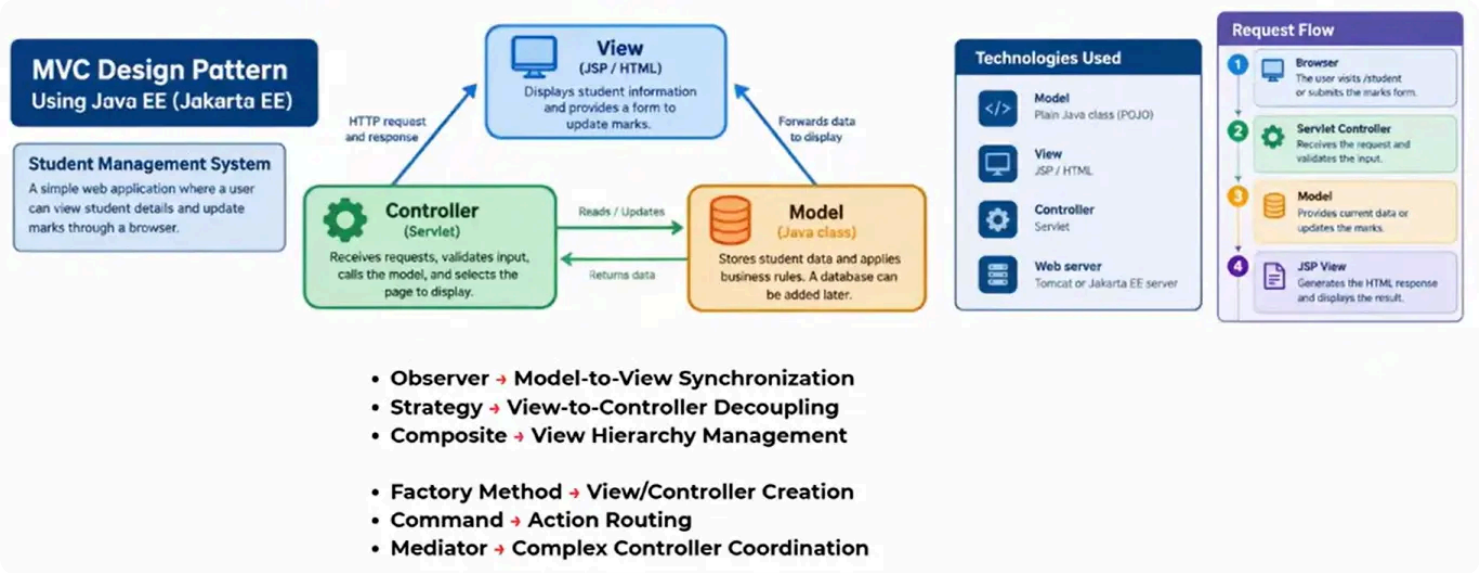
The doGet() or doPost() method receives the request. For a form submission, it reads and validates the marks

Step 3 - Model

The Student object provides the current details or updates the marks if the input is valid.

Step 4 - JSP View

The Servlet puts the student object into the request and forwards to student.jsp, which generates the HTML response.



Anti-Patterns

What are Design patterns

A design pattern is a reusable, well-established approach to solving a recurring software design problem in an appropriate context.

For example, the Strategy pattern lets us switch between different algorithms without rewriting the class that uses them.

```
interface PaymentStrategy{
    void pay(double amount);
}

class CardPayment implements PaymentStrategy{
    public void pay(double amount){
        System.out.println("Paid by card:" + amount);
    }
}
```

We use a design pattern because it can help use produce software that is easier to extend, maintain, and understand.

What are Anti-Patterns

An anti-pattern is a recurring approach to a software development problem that appears convenient but tends to produce harmful consequences over time.

For example, placing student registration, fee calculation, exam processing, and report generation in one enormous class might seem convenient initially. As the system grows, however, that class becomes difficult to test and maintain.

Design Pattern	Anti-Pattern
• A reusable approach that can solve a recurring problem	• A recurring approach that tends to create problems
• Encourages an appropriate structure or collaboration	• Often creates unnecessary complexity, dependencies, or risks
• Helps guide design decisions	Helps identify and correct poor design decisions
• Example: Strategy	• Example: God Object

Anti-Pattern 1: Spaghetti Code

Spaghetti Code is code with tangles control flow and poorly organized logic that makes it difficult to understand and safely modify.

Anti-Pattern 2: God Object

A God Object is a class that knows too much or does too much. It takes on many responsibilities that would be better distributed among appropriately designed classes.

Anti-Pattern 3: Copy-Paste Programming

Copy-Paste Programming occurs when developers duplicate existing code instead of reusing or appropriately extracting shared logic.

Anti-Pattern 4: Long Method

A Long Method is a method that contains so many steps, decisions, or responsibilities that it becomes difficult to understand, test, reuse, and maintain.

Anti-Pattern 5: Tight Coupling

One class depends too directly on another concrete class, making changes and testing difficult.

Anti-Pattern 6: Primitive Obsession

A design relies excessively on primitive types such as int, double, and String to represent meaningful domain concepts.

Anti-Pattern 7: Magic Numbers and Strings

Values appear directly in the code without clear names or explanations of their meaning.

Anti-Pattern 8: Swallowed Exceptions

A program catches an exception but ignores it, making failures difficult to detect and diagnose.